WEEK 4 · LESSON 20 OF 24 · HANDOUT

Documentation: Model Cards and Datasheets

Study time: **about 50 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=dQHjntrqrFQ — open the lesson online to watch inside the course.

🎓 **Go deeper:** PyGotham 2019: Ethics & Bias Detection in the Real World — https://www.youtube.com/watch?v=FkCb7345Hj8

🎓 **Go deeper:** PyCon US 2024: Trust Fall — Three Hidden Gems in MLflow — https://www.youtube.com/watch?v=cAgrIilgP4A

Your goal in this lesson: Write a datasheet for a dataset and a model card for a model, generate them automatically from real metrics, and keep them up to date with every release.

Why documentation?

Imagine a new developer, an auditor or a regulator in two years. They need to know: what is this model for, what data built it, how good is it, for whom does it work less well, and what must it **not** be used for? Two widely used formats answer these questions:

- **Datasheets for Datasets** (Gebru and colleagues, 2018): how a dataset was collected, what it contains, its limits and allowed uses.
- **Model Cards for Model Reporting** (Mitchell and colleagues, 2019): a short document about one model — intended use, performance broken down by group, limitations and ethical considerations.

Hugging Face shows a model card on every model page — you have seen them in Course 6.

```
import numpy as np
import pandas as pd

def make_applicants(n=3000, seed=7, drift=0.0, concept=0.0):
    # Made-up SACCO loan applications. drift > 0: incomes, loans and late payments change.
    # concept > 0: the link to repayment changes (salaried jobs protect less, late payments
hurt more).
    rng = np.random.default_rng(seed)
    gender = rng.choice(["female", "male"], n)
    area = rng.choice(["urban", "rural"], n, p=[0.6, 0.4])
    rural = area == "rural"
    income = (rng.lognormal(12.4, 0.5, n) * np.where(rural, 0.7, 1.0) * (1 - drift *
0.25)).round(-3)
    loan = (rng.uniform(50_000, 600_000, n) * (1 + drift)).round(-3)
    years = rng.integers(0, 13, n)
    late = rng.poisson(0.7 + drift, n)
    salaried = (rng.random(n) < np.where(rural, 0.3, 0.6)).astype(int)
    momo = np.clip(rng.normal(np.where(rural, 18, 45), 10), 0, None).round().astype(int)
    score = 3.0 - 1.6 * (loan / income) + 0.12 * years - 0.7 * late + 0.6 * salaried + 0.01 *
momo
    score = score - concept * (1.5 * salaried + 0.6 * late - 0.04 * (momo - 30))
    repaid = (rng.random(n) < 1 / (1 + np.exp(-score))).astype(int)
    officer = score - 0.9 * rural - 0.6 * (gender == "female") + rng.normal(0, 0.6, n)
    return pd.DataFrame({
        "applicant_id": np.arange(1, n + 1), "age": rng.integers(20, 63, n), "gender": gender,
"area": area,
        "income_rwf": income, "loan_rwf": loan, "years_member": years, "late_payments": late,
        "salaried": salaried, "momo_tx_month": momo,
        "approved_before": (officer > 1.2).astype(int), "repaid": repaid})

applicants = make_applicants()
print(applicants.shape, "| repaid:", round(applicants["repaid"].mean(), 3))
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

FEATURES = ["income_rwf", "loan_rwf", "years_member", "late_payments", "salaried",
"momo_tx_month"]
TARGET = "repaid"
train, test = train_test_split(applicants, test_size=0.25, random_state=0,
stratify=applicants[TARGET])
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
model.fit(train[FEATURES], train[TARGET])
proba = model.predict_proba(test[FEATURES])[:, 1]
print(f"accuracy {accuracy_score(test[TARGET], proba >= 0.5):.3f} | AUC
{roc_auc_score(test[TARGET], proba):.3f}")
```

A datasheet, generated from the data

```
def datasheet(df, name):
    lines = [f"# Datasheet: {name}", "",
             "## Motivation", "Created to train and audit a loan-repayment model for a SACCO
(practice data, no real people).", "",
             "## Composition", f"- Rows: {len(df):,}", f"- Columns: {len(df.columns)}"]
    for col in df.columns:
        s = df[col]
        if pd.api.types.is_numeric_dtype(s):
            info = f"min {s.min():,.0f}, median {s.median():,.0f}, max {s.max():,.0f}"
        else:
            info = ", ".join(f"{k} {v:.0%}" for k, v in s.value_counts(normalize=True).items())
        lines.append(f"- `{col}`: {info}; missing {s.isna().mean():.0%}")
    lines += ["", "## Known issues",
              "- `approved_before` records past officer decisions that were unfair to rural
applicants and women.",
              "- Rural applicants have lower incomes and fewer salaried jobs; several features
act as proxies for area.",
              "", "## Allowed uses", "Training and auditing credit-risk models with human
review. Not for marketing or for decisions without review.",
              "", "## Personal data", "Contains no names or contact details. Gender, area and
age are kept for fairness audits only."]
    return "\n".join(lines)

sheet = datasheet(applicants, "SACCO loan applications v1")
open("DATASHEET.md", "w").write(sheet)
print(sheet)
```

A model card, generated from the metrics

```
def group_metrics(data, group, approved):
    out = []
    for name, g in data.assign(approved=approved).groupby(group):
        out.append(f"| {group} = {name} | {len(g)} | {g['approved'].mean():.1%} | "
                   f"{g.loc[g[TARGET] == 1, 'approved'].mean():.1%} | {roc_auc_score(g[TARGET],
g['score']):.3f} |")
    return out

scored = test.assign(score=model.predict_proba(test[FEATURES])[:, 1])
approved = (scored["score"] >= 0.5).astype(int)

card = "\n".join([
    "# Model card: loan-repayment v1.0.0", "",
    "## Intended use",
    "- Supports SACCO loan officers by estimating the probability that an applicant repays.",
    "- Output: approve, or refer to an officer. **The model never refuses a loan on its
own.**", "",
    "## Out-of-scope uses",
    "- Fully automatic refusals; marketing; employment or insurance decisions; applicants
outside the SACCO.", "",
    "## Model", f"- Logistic regression on standardised features: {', '.join(FEATURES)}.",
    "- Gender, area and age are NOT inputs.", "",
    "## Data", "- Trained on 2,250 applications; tested on 750. See DATASHEET.md.", "",
    "## Performance (test set)",
    f"- AUC {roc_auc_score(test[TARGET], scored['score']):.3f}, accuracy
{accuracy_score(test[TARGET], approved):.3f}", "",
    "## Performance by group",
    "| Group | People | Approval rate | Approved if would repay | AUC |", "|---|---|---|---|---
|",
    *group_metrics(scored, "area", approved), *group_metrics(scored, "gender", approved), "",
    "## Limitations and ethical considerations",
    "- Rural applicants are approved less often (disparate impact below 0.8); human review of
refused rural applicants is required.",
    "- Performance will fall under concept drift; outcomes are monitored monthly.",
    "- Mobile-money activity can be inflated by applicants; its effect is monitored.", "",
    "## Owners", "- System owner: SACCO credit manager. Developer: data team. Next review: in 3
months.",
])
open("MODEL_CARD.md", "w").write(card)
print(card)
```

Because the card is **generated by code** from the real metrics, it cannot become out of date: regenerate it in CI for every release (Lesson 7) and store it next to the model in the registry (Lesson 8).

What makes good documentation

- **Honest:** includes weaknesses and groups where the model works less well.
- **Specific:** numbers, dates, versions, owners — not "the model is fair".
- **For its readers:** a short summary for managers, details for auditors.
- **Versioned:** one card per model version.

Key terms

- **Datasheet** — documentation of a dataset's origin, contents, limits and uses.
- **Model card** — documentation of a model's use, performance by group, limits and ethics.

- **Intended use / out-of-scope use** — what the model is and is not for.
- **Disaggregated metrics** — performance reported separately for each group.

Remember this: Document datasets with datasheets and models with model cards, generate them from real metrics in every release, and be honest about limits.

Name: _____ Date: _____

Activity

Task 1

Add an "age band" section to the model card's group table.

Task 2

Add a "Change log" section listing what changed since the previous version.

Task 3

Write a five-sentence summary of the model card for the SACCO board.

Task 4

Find a model card on Hugging Face and list which of our sections it has and which it misses.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What is a model card?

   A. A data table
   B. A business card
   C. A graphics card
   D. A short document about a model's use, performance by group, limits and ethics

2. What does a datasheet describe?

   A. How a dataset was collected, what it contains, its limits and allowed uses
   B. A spreadsheet's colours
   C. Server specifications
   D. Model weights

3. Why generate the model card with code?

   A. It looks nicer
   B. To hide limits
   C. It always uses the real, current metrics and cannot go out of date
   D. Laws require Python

4. What is an out-of-scope use?

   A. A use the model is not designed or approved for
   B. A server error
   C. A test
   D. The main use

5. Why report performance by group?

   A. Groups never differ
   B. To make the card longer
   C. Average performance can hide groups for whom the model works less well
   D. It is faster

6. Which is a sign of good documentation?

   A. It hides weaknesses
   B. It says 'the model is fair' with no numbers
   C. It is honest, specific and versioned
   D. It is never updated

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.